AMERICAN INTERNATIONAL UNIVERSITY BANGLADESH (AIUB)

FACULTY OF SCIENCE & TECHNOLOGY

Course Title

INTRODUCTION TO DATABASE (2108)

Semester: Spring 2023-2024

Section: [O]

TITLE

Company Structure Management System

Supervised By

MD Sajid Bin Faisal

Submitted By: Group No:02

Name	ID
SAGOR CHANDRA	22-49847-3
CHANDRIMA DAM	22-49509-3
SHUVRA KANTI DAS	22-49496-3
MST. FAREHA RAHMAN	22-49828-3

TABLE OF CONTENTS

TOPICS		Page no.
Title Page		1
Table of Content		2
1.	Introduction	3
2.	Case Study	4
3.	ER Diagram	5
4.	Normalization	6-8
5.	Finalization	9-10
6.	Table Creation	11-20
7.	Data Insertion	21-25
8.	Query Test	26-36
9.	DB connection	36-46
10.	Conclusion	47-47

1. Introduction

A proficient company management system is fundamental for optimizing operations, coordinating resources, and amplifying productivity. At the heart of such a system lies a meticulously crafted database, pivotal for storing, retrieving, and managing diverse company data facets. These include employee particulars, project specifics, financial records, and beyond. Through the implementation of a robust database management system, enterprises can elevate efficiency, streamline decision-making processes, and uphold data integrity. This introduction aims to elucidate the pivotal components and advantages inherent in a comprehensive company management system driven by a proficient database infrastructure.

In addition to operational benefits, a well-structured company management system powered by a database fosters collaboration and communication among team members. With centralized access to relevant information, employees can collaborate more effectively, share knowledge, and coordinate tasks seamlessly across departments and teams.

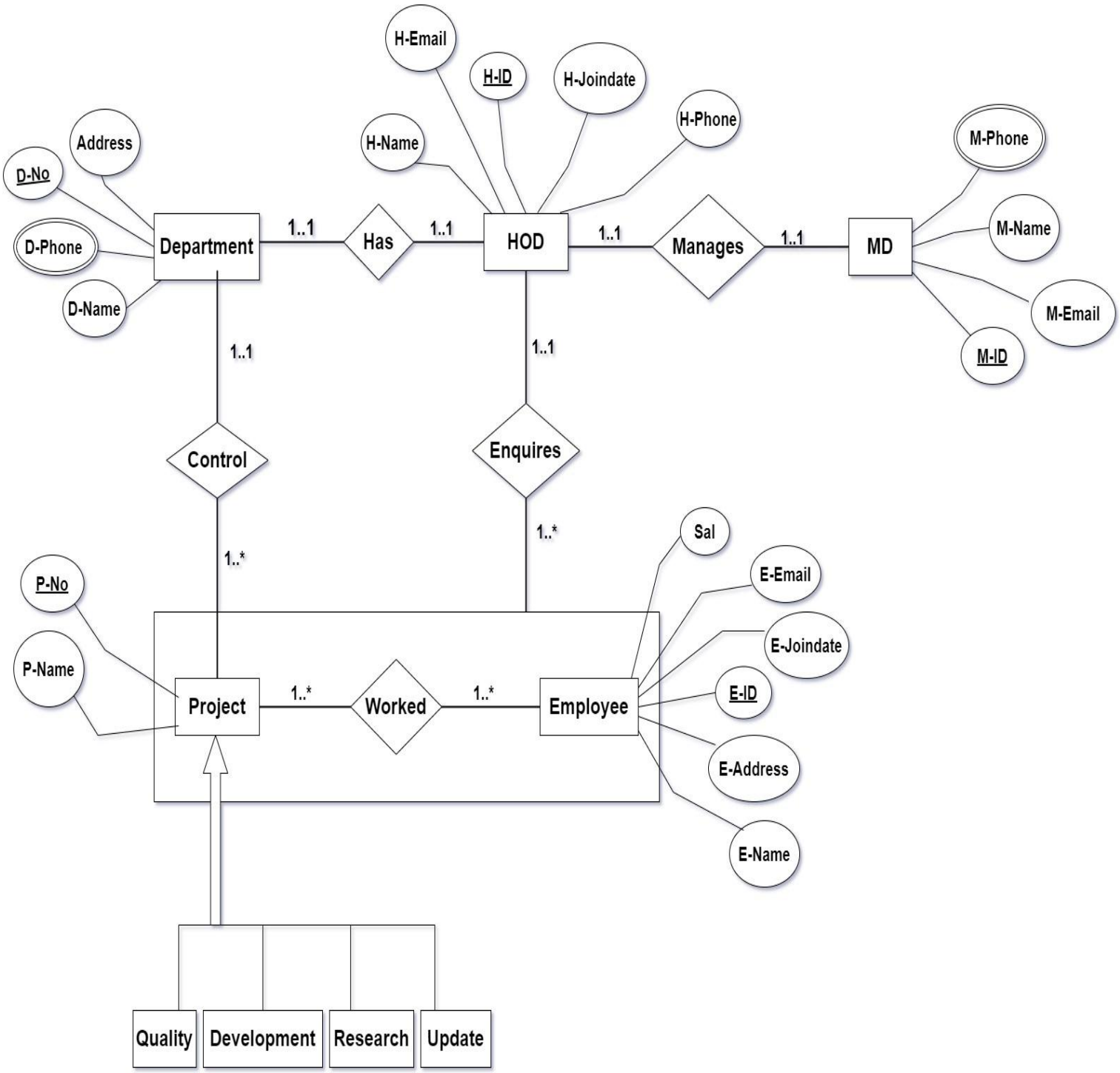
In summary, a comprehensive company management system supported by a robust database infrastructure is indispensable for modern businesses seeking to optimize their operations, make informed decisions, and stay competitive in today's dynamic marketplace.

2. Case Study / Scenario

StudentID1: 22-49496-3 Name: Shuvra Kanti Das	StudentID3: 22-49509-3 Name: Chandrima Dam
StudentID2: 22-49847-3 Name: Sagor Chandra	StudentID4: 22-49828-3 Name: Mst. Fareha Rahman
CO2: Understand the fundamental concepts underlying database systems and gain hands-on experience with ER diagram Case study	
PO-c2: Develop process for complex computer science and engineering problems considering cultural and societal factors.	Marks

In a company structure management system, the company is organized into departments. Each department is identified by a department ID number. The system also stores department name, department email address, and multiple phone numbers for each department. Each department has a Head of Department (HOD). The information of the HOD includes name, id, phone number, join date, address, and salary. The HOD is managed by a Managing Director (MD), who has information such as M-Name, M-email, M-mobile-no, and M-ID. Each department controls several projects aimed at improving the company. Each project has a p-name, and a p-No. Projects are specialized in development, research, quality, and updates. Every project is worked on by many employees. To identify an employee, the system stores E-ID along with E-email, E-address, E-salary, and E-Join-date. The Head of the Department inquiries about projects and employees.

3. ER Diagram


H-Email
H-ID
H-Joindate
H-Phone
H-Name
Address
D-No
D-Phone
D-Name
Department
1..1
Has
1..1
HOD
1..1
Manages
1..1
MD
M-Phone
M-Name
M-Email
M-ID
1..1
1..1
Control
Enquires
1..*
1..*
Sal
E-Email
E-Joindate
E-ID
E-Address
E-Name
P-No
P-Name
Project
1..*
Worked
1..*
Employee
Quality
Development
Research
Update

4. Normalization

HAS RELATION:

UNF: D-NAME, D-NO, ADDRESS, D-PHONE, H-ID, H-NAME, H-EMAIL, H-JOINDATE, H-PHONE.

1NF: D-NO(PK), D-NAME, ADDRESS, D-PHONE, H-ID(PK), H-NAME, H-EMAIL, H-JOINDATE, H-PHONE.

2NF:

i. D-NO(PK), D-NAME, ADDRESS, D-PHONE.
ii. H-ID(PK), H-NAME, H-EMAIL, H-JOINDATE, H-PHONE.
iii. D-NO(PK), H-ID(FK).

3NF: As same as 2NF.

MANAGE RELATION:

UNF: H-ID, H-EMAIL, H-JOINDATE, H-NAME, H-PHONE, M-ID, M-NAME, M-EMAIL, M-PHONE.

1NF: H-ID(PK), H-EMAIL, H-JOINDATE, H-NAME, H-PHONE, M-ID(PK), M-NAME, M-EMAIL, M-PHONE.

2NF:

i. H-ID(PK), H-NAME, H-EMAIL, H-JOINDATE, H-PHONE.
ii. M-ID(PK), M-NAME, M-EMAIL, M-PHONE.
iii. M-ID(PK), H-ID(FK).

3NF: As same as 2NF.

CONTROL RELATION:

UNF: D-NAME, D-NO, ADDRESS, D-PHONE, P-NO, P-NAME.

1NF: D-NO(PK), D-NAME, ADDRESS, D-PHONE, P-NO(PK), P-NAME.

2NF:

i. D-NO(PK), D-NAME, ADDRESS, D-PHONE.
ii. P-NO(PK), P-NAME, D-NO(FK).

3NF: As same as 2NF.

WORK RELATION:

UNF: P-NO, P-NAME, E-ID, E-EMAIL, E-NAME, E-JOINDTAE, E-ADDRESS, SAL.

1NF: P-NO(PK), P-NAME, E-ID(PK), E-EMAIL, E-NAME, E-JOINDTAE, E-ADDRESS, SAL.

2NF:

i. P-NO(PK), P-NAME.
ii. E-ID(PK), E-EMAIL, E-NAME, E-JOINDTAE, E-ADDRESS, SAL.
iii. P-NO(PK), E-ID(FK).

3NF: As same as 2NF.

ENQUIRE RELATION:

UNF: H-ID, H-EMAIL, H-JOINDATE, H-NAME, H-PHONE, P-NO, P-NAME, E-ID, E-EMAIL, E-NAME, E-JOINDTAE, E-ADDRESS, SAL.

1NF: H-ID(PK), H-EMAIL, H-JOINDATE, H-NAME, H-PHONE, P-NO(PK), P-NAME, E-ID(PK), E-EMAIL, E-NAME, E-JOINDTAE, E-ADDRESS, SAL.

2NF:

i. H-ID(PK), H-EMAIL, H-JOINDATE, H-NAME, H-PHONE
ii. P-NO(FK), P-NAME, E-ID(PK), E-EMAIL, E-NAME, E-JOINDTAE, E-ADDRESS, SAL, H-ID(FK)

3NF: As same as 2NF

5. Finalization

I. D-NO(PK), D-NAME, ADDRESS, D-PHONE.
II. H-ID(PK), H-NAME, H-EMAIL, H-JOINDATE, H-PHONE.
III. D-NO(PK), H-ID(FK).
IV. H-ID(PK), H-NAME, H-EMAIL, H-JOINDATE, H-PHONE.
V. M-ID(PK), M-NAME, M-EMAIL, M-PHONE.
VI. M-ID(PK), H-ID(FK).
VII. D-NO(PK), D-NAME, ADDRESS, D-PHONE.
VIII. P-NO(PK), P-NAME, D-NO(FK).
IX. P-NO(PK), P-NAME

X. E-ID(PK), E-EMAIL, E-NAME, E-JOINDTAE, E-ADDRESS, SAL.

XI. P-NO(PK), E-ID(FK).

XII. H-ID(PK), H-EMAIL, H-JOINDATE, H-NAME, H-PHONE

XIII. P-NO(FK), P-NAME, E-ID(PK), E-EMAIL, E-NAME, E-JOINDTAE, E-ADDRESS, SAL, H-ID(FK).

5.1 FINAL TABLES

I. D-NO(PK), D-NAME, ADDRESS, D-PHONE. **(DEPARTMENT)**

II. D-NO(PK), H-ID(FK). **(HAS)**

III. M-ID(PK), M-NAME, M-EMAIL, M-PHONE. **(MANAGER)**

IV. M-ID(PK), H-ID(FK). **(MANAGE)**

V. P-NO(PK), P-NAME, D-NO(FK). **(CONTROL)**

VI. P-NO(PK), P-NAME. **(PROJECT)**

VII. E-ID(PK), E-EMAIL, E-NAME, E-JOINDTAE, E-ADDRESS, SAL. **(EMPLOYEE)**

VIII. P-NO(PK), E-ID(FK). **(WORK)**

IX. H-ID(PK), H-EMAIL, H-JOINDATE, H-NAME, H-PHONE. **(HOD)**

X. P-NO(FK), P-NAME, E-ID(PK), E-EMAIL, E-NAME, E-JOINDTAE, E-ADDRESS, SAL, H-ID(FK). **(ENQUIRE)**

5.2 Final Table Name

i. Department

ii. Has

iii. Manager

iv. Manage

v. Control

vi. Project

vii. Employee

viii. Work

ix. Hod

x. Enquire

6. Table Creation (DDL Operations)

StudentID1: 22-49496-3 Name: Shuvra Kanti Das	StudentID3: 22-49509-3 Name: Chandrima Dam
StudentID2: 22-49847-3 Name: Sagor Chandra	StudentID4: 22-49828-3 Name: Mst. Fareha Rahman
CO4: Creating DML, DDL using Oracle and connection with ODBC/JDBC for existing JAVA application	
PO-e-2: Use modern engineering and IT tools for prediction and modeling of complex computer science and engineering problem	Marks

6.1 Creation Of Department Table:

User: COMPANYMNGT

Home > SQL > SQL Commands

Autocommit Display 100

```
create table department (DNo number (2) primary key,
DName varchar (10), DAdress varchar (20), DPhn number (15))
```

FIGURE – 6.1

6.2 Describe Of Department:

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
DEPARTMENT	DNO	Number	-	2	0	1	-	-	-
	DNAME	Varchar2	10	-	-	-	✓	-	-
	DADRESS	Varchar2	20	-	-	-	✓	-	-
	DPHN	Number	-	15	0	-	✓	-	-
									1 - 4

FIGURE – 6.2

6.3 Creation Of Has Table:

User: COMPANYMNGT

Home > SQL > SQL Commands

Autocommit Display 100

```
create table has (DNo number (2) primary key, HID number (2),
constraint hd foreign key (HID) references HOD (HID))
```

FIGURE – 6.3

6.4 Describe Of Has Table:

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
HAS	DNO	Number	-	2	0	1	-	-	-
	HID	Number	-	2	0	-	✓	-	-
									1 - 2

FIGURE – 6.4

6.5 Creation Of Manager Table:

User: COMPANYMNGT

Home > SQL > SQL Commands

Autocommit Display 100

```
create table manager (MID number (2) primary key,
Mname varchar (10), Mmail varchar (12), MPhn number (15))
```

FIGURE – 6.5

6.6 Describe Of Manager Table:

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
MANAGER	MID	Number	-	2	0	1	-	-	-
	MNAME	Varchar2	10	-	-	-	✓	-	-
	MMAIL	Varchar2	12	-	-	-	✓	-	-
	MPHN	Number	-	15	0	-	✓	-	-
									1 - 4

FIGURE – 6.6

6.7 Creation Of Manage Table:

User: COMPANYMNGT

Home > SQL > SQL Commands

Autocommit Display 100

```
create table manage (MID number (2) primary key,
HID number (2), constraint mh foreign key (HID) references HOD (HID))
```

FIGURE – 6.7

6.8 Describe Of Manage Table:

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
MANAGE	MID	Number	-	2	0	1	-	-	-
	HID	Number	-	2	0	-	✓	-	-
									1 - 2

FIGURE – 6.8

6.9 Creation Of Control Table:

User: COMPANYMNGT

Home > SQL > SQL Commands

Autocommit Display 100

```
create table control (PNo number (2) primary key, PName varchar (10),
DNo number (2), constraint dp foreign key (DNo) references department (DNo))
```

FIGURE – 6.9

6.10 Describe Of Control Table:

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
CONTROL	PNO	Number	-	2	0	1	-	-	-
	PNAME	Varchar2	10	-	-	-	✓	-	-
	DNO	Number	-	2	0	-	✓	-	-
									1 - 3

FIGURE – 6.10

6.11 Creation Of Project Table:

User: COMPANYMNGT

Home > SQL > SQL Commands

Autocommit Display 100

```
create table project (PNo number (2) primary key,
PName varchar (10))
```

FIGURE – 6.11

6.12 Describe Of Project Table:

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
PROJECT	PNO	Number	-	2	0	1	-	-	-
	PNAME	Varchar2	10	-	-	-	✓	-	-
									1 - 2

FIGURE – 6.12

6.13 Creation Of Employee Table:

User: COMPANYMNGT

Home > SQL > SQL Commands

Autocommit Display 100

```
create table employee (EID number (2)primary key, Ejoindate date,
EName varchar (10), Email varchar (12), Esal number (8), EAdress varchar (20))
```

FIGURE – 6.13

6.14 Describe Of Employee Table:

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
EMPLOYEE	EID	Number	-	2	0	1	-	-	-
	EJOINDATE	Date	7	-	-	-	✓	-	-
	ENAME	Varchar2	10	-	-	-	✓	-	-
	EMAIL	Varchar2	12	-	-	-	✓	-	-
	ESAL	Number	-	8	0	-	✓	-	-
	EADRESS	Varchar2	20	-	-	-	✓	-	-
									1 - 6

FIGURE – 6.14

6.15 Creation Of Work Table:

User: COMPANYMNGT

Home > SQL > SQL Commands

Autocommit Display 100

```
create table work (PNo number (2) primary key, EID number (2),
constraint ep foreign key (EID) references employee (EID))
```

FIGURE – 6.15

6.16 Describe Of Work Table:

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
WORK	PNO	Number	-	2	0	1	-	-	-
	EID	Number	-	2	0	-	✓	-	-
									1 - 2

FIGURE – 6.16

6.17 Creation Of HOD Table:

User: COMPANYMNGT

Home > SQL > SQL Commands

Autocommit Display 100

```
create table HOD (HID number (2) primary key,
HName varchar (10), Hmail varchar (12), HPhn number (15), Hjoindate date)
```

FIGURE – 6.17

6.18 Describe Of HOD Table:

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
HOD	HID	Number	-	2	0	1	-	-	-
	HNAME	Varchar2	10	-	-	-	✓	-	-
	HMAIL	Varchar2	12	-	-	-	✓	-	-
	HPHN	Number	-	15	0	-	✓	-	-
	HJOINDATE	Date	7	-	-	-	✓	-	-

1 - 5

FIGURE – 6.18

6.19 Creation Of Enquire Table:

User: COMPANYMNGT

Home > SQL > SQL Commands

Autocommit Display 100

```
create table enquire (HID number (2), EID number (2) primary key,
EName varchar (10),Email varchar (12), Esal number (8),
EAdress varchar (20), PNo number (2), constraint ghk foreign key (PNo)
references project (PNo), constraint phk foreign key (HID) references HOD (HID))
```

FIGURE – 6.19

6.20 Describe Of Enquire Table:

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
ENQUIRE	HID	Number	-	2	0	-	✓	-	-
	EID	Number	-	2	0	1	-	-	-
	ENAME	Varchar2	10	-	-	-	✓	-	-
	EMAIL	Varchar2	12	-	-	-	✓	-	-
	ESAL	Number	-	8	0	-	✓	-	-
	EADRESS	Varchar2	20	-	-	-	✓	-	-
	PNO	Number	-	2	0	-	✓	-	-
									1 - 7

FIGURE – 6.20

7. Inserted Values in the table

7.1 Values Of Department Table :

Results Explain Describe Saved SQL History

DNO	DNAME	DADRESS	DPHN
10	Marketing	Dhaka	783250936
20	IT	Comilla	783250946
30	HR	Narayanganj	783250938
40	Management	Kulna	783259936
50	Finance	Sylhet	757353226

5 rows returned in 0.00 seconds CSV Export

FIGURE – 7.1

7.4 Values Of Has Table :

Results Explain Describe Saved SQL History

DNO	HID
10	15
20	25
30	35
40	75

4 rows returned in 0.00 seconds CSV Export

FIGURE – 7.2

7.3 Values Of Manager Table :

Results Explain Describe Saved SQL History

MID	MNAME	MMAIL	MPHN
11	TRVIS	tr@gmail.com	1677638822
22	PERRY	pr@gmail.com	1677638822
33	JOHN	jh@gmail.com	1677638822
44	MACC	ma@gmail.com	1677638822

4 rows returned in 0.00 seconds CSV Export

FIGURE – 7.3

7.8 Values Of Manage Table :

Results Explain Describe Saved SQL History

MID	HID
11	15
22	25
33	75
44	35

4 rows returned in 0.00 seconds CSV Export

FIGURE – 7.4

7.5 Values Of Control Table :

Results Explain Describe Saved SQL History

PNO	PNAME	DNO
10	Local Link	10
20	Unity Hub	20
30	Safe Zone	30
40	EcoPower	40
50	Urban Link	50

5 rows returned in 0.00 seconds CSV Export

FIGURE – 7.5

7.6 Values Of Project Table :

Results Explain Describe Saved SQL History

PNO	PNAME
50	Urban Link
10	Local Link
20	Unity Hub
30	Safe Zone
40	EcoPower

5 rows returned in 0.00 seconds CSV Export

FIGURE – 7.6

7.7 Values Of Employee Table :

Results Explain Describe Saved SQL History

EID	EJOINDATE	ENAME	EMAIL	ESAL	EADRESS
30	01-MAR-08	ALEX CARRY	x@gmail.com	12000	BARISAL
40	23-SEP-18	MARK WOOD	a@gmail.com	30000	MOULVIBAZAR
50	10-AUG-09	TIM DAVID	t@gmail.com	4000	CHITTAGONG
60	17-NOV-15	MARRY COM	m@gmail.com	75000	MYMENSINGH
70	30-APR-23	PAT MILLS	p@gmail.com	6000	GAZIPUR

5 rows returned in 0.00 seconds CSV Export

FIGURE – 7.7

7.8 Values Of Work Table :

Results Explain Describe Saved SQL History

PNO	EID
10	30
20	40
30	50
40	60
50	70

5 rows returned in 0.00 seconds CSV Export

FIGURE – 7.8

7.9 Values Of HOD Table :

Results Explain Describe Saved SQL History

HID	HNAME	HMAIL	HPHN	HJOINDATE
75	CHANDRIMA	cd@yahoo.com	98003776383	12-FEB-02
25	FAREHA	fr@gmail.com	90763462163	30-MAR-14
35	SHUVRO	sk@gmail.com	98078350098	21-MAY-10
15	SAGOR	cd@gmail.com	95735342773	01-FEB-08

4 rows returned in 0.02 seconds CSV Export

FIGURE – 7.9

7.10 Values Of Enquire Table :

Results Explain Describe Saved SQL History

HID	EID	ENAME	EMAIL	ESAL	EADRESS	PNO
15	30	ALEX CARRY	x@gmail.com	12000	BARISAL	10
25	40	MARK WOOD	a@gmail.com	30000	MOULVIBAZAR	20
35	50	TIM DAVID	t@gmail.com	4000	CHITTAGONG	30
75	60	MARRY COM	m@gmail.com	75000	MYMENSINGH	40

4 rows returned in 0.00 seconds CSV Export

FIGURE – 7.10

8. Query Test

8.1 Simple Query

➔ **Show department name and department number from the department table.**

User: COMPANYMNGT

Home > SQL > SQL Commands

Autocommit Display 100

```
select dname, dno from department
```

FIGURE – 8.1

Results Explain Describe Saved SQL History

DNAME	DNO
Marketing	10
IT	20
HR	30
Management	40
Finance	50

5 rows returned in 0.02 seconds CSV Export

FIGURE – 8.2

8.2 Single Row Function

➔ **Show the ID, name, and email address for managers named 'John'.**

User: COMPANYMNGT

Home > SQL > SQL Commands

Autocommit Display 10

```
SELECT  Mid, Mname, Mmail
FROM    manager
WHERE   LOWER(Mname) = 'john'
```

FIGURE – 8.3

Results Explain Describe Saved SQL History

MID	MNAME	MMAIL
33	JOHN	jh@gmail.com

1 rows returned in 0.00 seconds CSV Export

FIGURE – 8.4

8.3 Multiple Row Function

➔ **Show the names, the length of the names and the position of the first occurrence of the letter 'A' in the names from the enquire table.**

User: COMPANYMNGT

Home > SQL > SQL Commands

☑ Autocommit Display 10

```
SELECT Ename, LENGTH(Ename),
INSTR(Ename, 'A')FROM enquire
```

FIGURE – 8.5

Results Explain Describe Saved SQL History

ENAME	LENGTH(ENAME)	INSTR(ENAME,'A')
ALEX CARRY	10	1
MARK WOOD	9	2
TIM DAVID	9	6
MARRY COM	9	2

4 rows returned in 0.02 seconds CSV Export

FIGURE – 8.6

8.4 Group Function

➔ **Show maximum salary and minimum salary from enquire table.**

User: COMPANYMNGT

Home > SQL > SQL Commands

Autocommit Display 10

```
select MIN(Esal), MAX(Esal) from enquire
```

FIGURE – 8.7

Results Explain Describe Saved SQL History

MIN(ESAL)	MAX(ESAL)
4000	75000

1 rows returned in 0.00 seconds CSV Export

FIGURE – 8.8

8.5 Single Row Subquery

➔ **Show the ename and joindate from employee table whose esal is less than eid = 70**

User: COMPANYMNGT

Home > SQL > SQL Commands

Autocommit Display 10

```
SELECT ename, ejoindate
FROM employee
WHERE Esal < (SELECT Esal FROM employee WHERE EID = 70);
```

FIGURE – 8.9

Results Explain Describe Saved SQL History

ENAME	EJOINDATE
TIM DAVID	10-AUG-09

1 rows returned in 0.00 seconds CSV Export

FIGURE – 8.10

8.6 Multiple Row Subquery

➔ **Show the eid and joindate from employee table where esal less than all of the 'ALEX CARRY'.**

User: COMPANYMNGT

Home > SQL > SQL Commands

Autocommit Display 10

```
SELECT Eid, Ejoindate
FROM employee
WHERE Esal < ALL (SELECT Esal FROM employee WHERE Ename LIKE 'ALEX CARRY');
```

FIGURE – 8.11

Results Explain Describe Saved SQL History

EID	EJOINDATE
50	10-AUG-09
70	30-APR-23

2 rows returned in 0.02 seconds CSV Export

FIGURE – 8.12

8.7 Self Join

➔ **Show a self-join in the Department table where it will show name and address within itself.**

User: COMPANYMNGT

Home > SQL > SQL Commands

Autocommit Display 50

```
SELECT a.dname||' department is located in '||b.dadress
FROM    department a, department b
WHERE a.dno = b.dno;
```

FIGURE – 8.13

Results Explain Describe Saved SQL History

A.DNAME||'DEPARTMENTISLOCATEDIN'||B.DADRESS
Marketing department is located in Dhaka
IT department is located in Comilla
HR department is located in Narayanganj
Management department is located in Kulna
Finance department is located in Sylhet

5 rows returned in 0.01 seconds CSV Export

FIGURE – 8.14

8.8 Outer Join

➔ **Show pname, pno, eid from the project and work table according to the order (default) of pno.**

User: COMPANYMNGT

Home > SQL > SQL Commands

☑ Autocommit Display 200

```
SELECT  p.pname, w.pno, w.eid FROM project p, work w
WHERE   p.pno(+) = w.pno ORDER By w.pno
```

FIGURE – 8.15

Results Explain Describe Saved SQL History

PNAME	PNO	EID
Local Link	10	30
Unity Hub	20	40
Safe Zone	30	50
EcoPower	40	60
Urban Link	50	70

5 rows returned in 0.01 seconds CSV Export

FIGURE – 8.16

8.9 Simple View

➔ **Create a view named as hodvu30 and show the id, name, join date of the HOD where hid is 15.**

User: COMPANYMNGT

Home > SQL > SQL Commands

☑ Autocommit Display 100

```
create view hodvu30 as select hid, hname, hjoindate
from hod where hid = 15
```

FIGURE – 8.17

Results Explain Describe Saved SQL History

Object Type **VIEW** Object **HODVU30**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
HODVU30	HID	Number	-	2	0	-	-	-	-
	HNAME	Varchar2	10	-	-	-	✓	-	-
	HJOINDATE	Date	7	-	-	-	✓	-	-
									1 - 3

FIGURE – 8.18: Description of the simple view

Results Explain Describe Saved SQL History

HID	HNAME	HJOINDATE
15	SAGOR	01-FEB-08

1 rows returned in 0.00 seconds CSV Export

FIGURE – 8.19: Simple view as a whole table

8.10 Complex View

➔ **Create a view named compview10 (SUMSAL) as select sum (esal) from enquire where ename like '%A%'.**

User: COMPANYMNGT

Home > SQL > SQL Commands

Autocommit Display 100

```
create view compview10 (sumsal) as select sum(esal)
FROM enquire WHERE ename like '%A%'
```

FIGURE – 8.20

Results Explain Describe Saved SQL History

Object Type **VIEW** Object **COMPVIEW10**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
COMPVIEW10	SUMSAL	Number	-	-	-	-	✓	-	-

1 - 1

FIGURE – 8.21: Description of the complex view

Results Explain Describe Saved SQL History

SUMSAL
121000

1 rows returned in 0.00 seconds CSV Export

FIGURE – 8.22: Complex view as a whole table

9. DB Connection

DB Connection -1 :

SAGOR CHANDRA (Table Name: **Manager**)\

1.First, install the MySQL Java Connector by downloading the Maven package containing the JAR file version 8.4.0.

2.Download and install XAMPP, which includes Apache, MariaDB, Perl, PHP, and MySQL servers. Start the Apache and MySQL servers, then open the MySQL server admin panel.

3.In the admin panel, create a database called "companymngt" and a table named "manager" with four columns. Insert values into the table, adding five rows in total.

Here is MySQL table

MID	MName	Mmail	Mphn
11	TRAVIS	tr@gmail.com	1677638822
22	PERRY	pr@gmail.com	1677638822
33	JOHN	jh@gmail.com	1677638822
44	MACC	ma@gmail.com	1677638822

FIGURE – 9.1 Output of the table from the MySQL server.

4. Download and install the Apache NetBeans IDE21. Then, add the JAR file to the project's library section in the IDE.

5. Write the DB connection code inside the "manager" class of the project. The code involves:

- Registering the MySQL JDBC driver dynamically using Class.forName("com.mysql.cj.jdbc.Driver"). JDBC drivers are essential for connecting Java applications with databases.
- Establishing a database connection using the DriverManager and obtaining a connection object.
- Creating a statement object using the connection's createStatement method.
- Executing SQL queries and obtaining result sets using the statement object.
- Closing the database connection using the connection.close() method for proper resource management.
- The **Manager** class is run.

The Main Code –

```java
import java.sql.*;
public class Manager {
    public static void main(String[] args) {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            Connection connection = DriverManager.getConnection("jdbc:mysql://localhost:3306/companymngt", "root", "");
            System.out.println("Connected");
            Statement statement = connection.createStatement();
            ResultSet resultSet = statement.executeQuery("SELECT * FROM `manager`;");
            while (resultSet.next()) {
                System.out.print("MID = " + resultSet.getInt(1) + ", ");
                System.out.print("MName = " + resultSet.getString(2) + ", ");
                System.out.print("Mmail = " + resultSet.getString(3) + ", ");
                System.out.println("Mphn  = " + resultSet.getInt(4));
              }

            connection.close();
        } catch (Exception ex) {
            System.out.println(ex);
        }
    }
}
```

FIGURE – 9.2 Java code to connect MySQL database and to show the output.

The Output-

```
Output - manager (run)
run:
Connected
MID = 11, MName = TRAVIS, Mmail = tr@gmail.com, Mphn  = 1677638822
MID = 22, MName = PERRY, Mmail = pr@gmail.com, Mphn  = 1677638822
MID = 33, MName = JOHN, Mmail = jh@gmail.com, Mphn  = 1677638822
MID = 44, MName = MACC, Mmail = ma@gmail.com, Mphn  = 1677638822
BUILD SUCCESSFUL (total time: 0 seconds)
```

FIGURE – 9.3 Output of the query from the code.

DB Connection -2 :
Fareha Rahman (Table Name : **Department**)
1. First go to MySQL java connector file .
2. Then search and install xampp . After installed we have to go xampp control and start Apace and MySQL.
3. Go to localhost and create a new table with table name and insert the values in the table.
The table in MySQL:

DNO	DNAME	DADDRESS	DPHONE
10	Marketing	Dhaka	783250936
20	IT	Comilla	783250946
30	HR	Narayanganj	783250938
40	Management	Kulna	783259936
50	Finance	Sylhet	757353226

FIGURE – 9.4 Output of the table from the MySQL server.

4. Chosen a Integrated Development Environment (IDE) of my choice. I used Netbense code for Java development.

5. In my Java code, loaded MySQL. Java Connector driver by imporing the JAR file into my project and using the Class.forName() method to register it.

6. Used the DriverManager.gorConnection() method to establish a connection to my MySQL database by providing the appropriate URL, username, and password. My usernanie root and there was no password set.

7. After establishing the connection, created a Statement or CrentedStatement object to execute SQL queries .

8. Used the crusted statement 10 exusate SOL queries like SELECT, INSERT,UPDATE, or DELETE. Captured the necessary results.

9. Used the ResultSet object to retrieve and process the data

10. Performed necessary operations on the dats retrieved from the database.

The Main Code –

```java
import java.sql.*;
public class Dbconnection {
public static void main(String[] args) {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            Connection connection = DriverManager.getConnection("jdbc:mysql://localhost:3306/department", "root",
            System.out.println("Connected");
            Statement st = connection.createStatement();
            ResultSet resultSet = st.executeQuery("SELECT * FROM `department`;");
            while (resultSet.next()) {
                System.out.print("DNO   = " + resultSet.getInt(1) + ", ");
                System.out.print("DNAME = " + resultSet.getString(2) + ", ");
                System.out.print("DADDRESS = " + resultSet.getString(3) + ", ");
                System.out.println("DPHONE = " + resultSet.getInt(4));
            }
           resultSet.close();
            st.close();
            connection.close();
        } catch (ClassNotFoundException | SQLException e) {
                    e.printStackTrace();
        }
```

FIGURE – 9.5 Java code to connect MySQL database and to show the output.

The Output-

```
Connected
DNO   = 10, DNAME = Marketing, DADDRESS = Dhaka, DPHONE = 783250936
DNO   = 20, DNAME = IT, DADDRESS = Comilla, DPHONE = 783250946
DNO   = 30, DNAME = HR, DADDRESS = Narayanganj, DPHONE = 783250938
DNO   = 40, DNAME = Management, DADDRESS = Kulna, DPHONE = 783259936
DNO   = 50, DNAME = Finance, DADDRESS = Sylhet, DPHONE = 757353226
BUILD SUCCESSFUL (total time: 3 seconds)
```

FIGURE – 9.6 Output of the query from the code.

DB Connection -3 :

Chandrima Dam (Table Name : **HOD**)

1. Choosing the IDE:

- For Java development, NetBeans IDE is selected.

2. Downloading MySQL Connector/J JDBC Driver:

- The MySQL Connector/J JDBC driver is downloaded from the official MySQL website, and the mysql-connector-j-8.4.0.jar file is extracted from the downloaded zip file.

3. Setting up XAMPP:

- XAMPP, a free and open-source web server solution stack developed by Apache Friends, is downloaded and installed.
- The XAMPP control panel is launched, ensuring that both the Apache and MySQL modules are running.
- The admin button beside the start column is clicked to open the PhpMyAdmin interface.

4. Creating a Database and Table in PhpMyAdmin:

- In the PhpMyAdmin interface, a new database named companymngt is created.
- Within the companymngt database, a new table named employee is created with columns HID (INT), Hname (VARCHAR), Hmail (VARCHAR), EPhn (INT), and Ejoindate (DATE). The SQL query to create the table is executed.

5. Setting up NetBeans IDE:

- NetBeans IDE is opened.
- Within the project, a package named employee is created.
- Inside the employee package, a Java class named Employee is created.
- The provided Java code is copied and pasted into the Employee.java file.

6. Adding MySQL Connector/J to the Project:

- The mysql-connector-j-8.4.0.jar file is added to the project's classpath:
- Right-click on the project name in the Project Explorer.
- Select "Properties" -> "Libraries" -> "Add JAR/Folder".
- Navigate to the location of the Connector/J JAR file and select it.

7. Updating MySQL Connection Information:

- If necessary, update the MySQL connection URL, username, and password in the DriverManager.getConnection() method call.

8. Running the Application:

- The HOD class is executed to run the application.

The table in mysql:

HID	HNAME	HMAIL	HPHN	HJOINDATE
15	SAGOR	cd@gmail.com	957534273	2002-02-12
25	FAREHA	fr@gmail.com	907346163	2014-03-30
35	SHUVRO	sk@gmail.com	980785098	2010-05-21
75	CHANDRIMA	cd@yahoo.com	980376383	2008-02-01

FIGURE – 9.7 Output of the table from the MySQL server.

The Main Code –

```
1   package hod;
2   import java.sql.Connection;
3   import java.sql.DriverManager;
4   import java.sql.ResultSet;
    import java.sql.SQLException;
6   import java.sql.Statement;
7   public class Hod {
8   public static void main(String[] args) {
9    try {
10   Class.forName("com.mysql.cj.jdbc.Driver");
     Connection conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/companymngt", "root", "");
12   System.out.println("Connected");
13   Statement st = conn.createStatement();
14   ResultSet resultSet = st.executeQuery("SELECT * FROM hod;");
15  while (resultSet.next()) {
16   System.out.print  ("HID   = " + resultSet.getInt(1)    + ", ");
17   System.out.print  ("HName = " + resultSet.getString(2) + ", ");
```

```
18   System.out.print  ("Hmail = " + resultSet.getString(3) + ", ");
19   System.out.println("Hphn  = " + resultSet.getInt(4)   + ", ");
20   System.out.println ("Hjoindate = " + resultSet .getDate(5));
21       }
22     conn.close();
     } catch (Exception s) {
24           System.out.println(s);
25       }
26    }
```

FIGURE – 9.8 Java code to connect MySQL database and to show the output.

The Output-

```
HID   = 15, HName = SAGOR, Hmail = cd@gmail.com, Hphn  = 957534273,
Hjoindate = 2002-02-12
HID   = 25, HName = FAREHA, Hmail = fr@gmail.com, Hphn  = 907346163,
Hjoindate = 2014-03-30
HID   = 35, HName = SHUVRO, Hmail = sk@gmail.com, Hphn  = 980785098,
Hjoindate = 2010-05-21
HID   = 75, HName = CHANDRIMA, Hmail = cd@yahoo.com, Hphn  = 980376383,
Hjoindate = 2008-02-01
BUILD SUCCESSFUL (total time: 0 seconds)
```

FIGURE – 9.9 Output of the query from the code.

DB Connection -4:
Shuvra Kanti Das (Table Name : **Employee**)

1. An IDE such as Eclipse, IntelliJ IDEA, or NetBeans is chosen for Java development. For this case study, NetBeans IDE is used.
2. The MySQL Connector/J JDBC driver is downloaded from the official MySQL website, and the downloaded zip file is extracted to find the **mysql-connector-j-8.4.0.jar** file.
3. XAMPP, a free and open-source cross-platform web server solution stack package developed by Apache Friends, is downloaded, and installed.
4. The XAMPP control panel is started, and it is ensured that the Apache and MySQL modules are running. The admin button right beside the start column is pressed.
5. A web browser is opened, and **http://localhost/phpmyadmin** is navigated to access the PhpMyAdmin interface. In the PhpMyAdmin interface, a new database named **companymngt** is created. Within the **companymngt** database, a new table named **employee** is created with the following columns**:** **EID** (INT), **Ejoindate** (DATE), **Ename** (VARCHAR), **Email** (VARCHAR), **Esal** (INT), **Eadress** (VARCHAR). Then, the SQL query to create the table is executed.

The table in MySQL:

EID	EJOINDATE	ENAME	EMAIL	ESAL	EADRESS
30	2008-03-01	ALEX CARRY	x@gmail.com	12000	BARISAL
40	2018-09-23	MARK WOOD	a@gmail.com	30000	MOULVIBAZAR
50	2009-08-10	TIM DAVID	t@gmail.com	4000	CHITTAGONG
60	2015-11-17	MARRY COM	m@gmail.com	75000	MYMENSINGH
70	2023-04-30	PAT MILLS	p@gmail.com	6000	GAZIPUR

FIGURE – 9.10 Output of the table from the MySQL server.

The Main Code –

```java
package employee;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class Employee {
    public static void main(String[] args) {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            Connection conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/companymngt", "root", "");
            System.out.println("Connected");
            Statement st = conn.createStatement();
            ResultSet resultSet = st.executeQuery("SELECT * FROM employee;");
            while (resultSet.next()) {
                System.out.print("EID   = " + resultSet.getInt(1) + ", ");
                System.out.print("Ejoindate = " + resultSet.getDate(2) + ", ");
                System.out.print("Ename = " + resultSet.getString(3) + ", ");
                System.out.print("Email  = " + resultSet.getString(4) + ", ");
                System.out.println("Esal = " + resultSet.getInt(5) + ", ");
                System.out.println("Eadress = " + resultSet.getString(6));
            }
            conn.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

FIGURE – 9.11 Java code to connect MySQL database and to show the output.

The Output-

```
Output - employee (run)
run:
Connected
EID   = 30, Ejoindate = 2008-03-01, Ename = ALEX CARRY, Email  = x@gmail.com, Esal = 12000,
Eadress = BARISAL
EID   = 40, Ejoindate = 2018-09-23, Ename = MARK WOOD, Email  = a@gmail.com, Esal = 30000,
Eadress = MOULVIBAZAR
EID   = 50, Ejoindate = 2009-08-10, Ename = TIM DAVID, Email  = t@gmail.com, Esal = 4000,
Eadress = CHITTAGONG
EID   = 60, Ejoindate = 2015-11-17, Ename = MARY COMM, Email  = m@gmail.com, Esal = 75000,
Eadress = MYMENSINGH
EID   = 70, Ejoindate = 2023-04-23, Ename = PAT MILLS, Email  = p@gmail.com, Esal = 6000,
Eadress = GAZIPUR
BUILD SUCCESSFUL (total time: 0 seconds)
```

FIGURE – 9.12 Output of the query from the code.

5. The NetBeans IDE is opened, and within the project, a package named **employee** is created. Inside the **employee** package, a Java class named **Employee** is created. The provided Java code is copied and pasted into the **Employee.java** file. The MySQL Connector/J JAR file **(mysql-connector-j-8.4.0.jar**) is added to the project's class path:

6. The project name in the Project Explorer is right clicked. "Properties" -> "Libraries" -> "Add JAR/Folder" is selected. The location where the Connector/J JAR file is found is navigated to, and it is selected.

7. The MySQL connection URL, username, and password in the **DriverManager.getConnection()** method call are updated if necessary.

8. The **Employee** class is run.

Conclusion

In this project, we have done normalization and finalization from an ER diagram named “Company Structure Management System” which had five different relations among the entities. After getting the final tables we showed all the queries that are necessary to create those tables in our own schema. We also showed how to insert values in those tables and took their screenshots. After that we did some query tests. At last, we have created some tables in mysql server and connected it with an IDE to show that these tables can be accessed by a java code.